

Lab 2

[valid 2024-2025]

Starting from this week...:

- Comment and properly format the source code of your programs (otherwise: -0.5 points)
- Use the [naming conventions](#) for writing Java code. "Naming conventions make programs more understandable by making them easier to read." (otherwise: -0.5 points)
- Use the API documentation: [Java Platform, Standard Edition API Specification](#) (otherwise...)

Objects and Classes

An instance of the **Student Project Allocation Problem** involves *students* and *projects*. A student may be enrolled in at most one project. A project may be assigned to at most one student. Each student has a set of acceptable projects.

We consider the problem of allocating students to projects, such that each student receives one acceptable project.

Example: 4 students, 4 projects.

Student preferences
S1: (P1, P2)
S2: (P1, P3)
S3: (P3, P4)
S4: (P1, P4)

A solution for this example could be: [(S1:P2), (S2:P1), (S3:P3), (S4:P4)]

The main specifications of the application are:

Compulsory (1p)

- Create an object-oriented model of the problem. You should have (at least) the following classes: Student, Project.
 - A project may be of two *types*, theoretical or practical. Use an enum in order to implement this feature.
 - Each class should have appropriate constructors, getters and setters.
Use the [IDE features](#) for code generation, such as [generating getters and setters](#).
 - The toString method from the *Object* class must be properly overridden for all the classes.
Use the [IDE features](#) for code generation, for example (in NetBeans) press *Alt+Ins* or invoke the context menu, select "Insert Code" and then "toString()" (or simply start typing "toString" and then press *Ctrl+Space*).
 - Create and print on the screen an object of each class.
-

Homework (2p)

- Consider that a project is proposed by a *teacher*. Both students and teachers are *persons*, having a name and a date of birth. Each student has a registration number and each teacher has a list of proposed projects.
- Create classes that describe the *problem* and the *solution*.

- Override the *equals* method from the *Object* class, such as the problem should not allow adding the same student, teacher or project twice.
- Create a method in the *Problem* class that returns an array of all the persons involved, students or teachers.
- Implement a simple greedy algorithm for allocating projects to students.
- Write [doc comments](#) in your source code and generate the class documentation using [javadoc](#).

Use **arrays**, **not collections** in order to represent a set of elements!

Bonus (2p)

- Implement an algorithm, based on [Hall's Theorem](#), in order to verify if it is possible to allocate a project for every student.
- Generate large random instances of the problem, feasible or infeasible, and test your algorithm, measuring the running time and the used memory.

(Warning: No points are awarded unless the implementation can be clearly explained).

Resources

- [Slides](#)
- [Tutorial: Object-Oriented Programming Concepts](#)
- [Tutorial: Classes and Objects](#)
- [Java Language Specification: Classes](#)
- [Enum Types](#)
- [The Date-Time package](#)

Objectives

- Create a project containing multiple classes.
- Instantiate classes and manipulate objects.
- Understand the concepts of: object identity, object state, encapsulation, property, accessors/mutators.
- Override methods of the *Object* class.
- Understand uni- and bi-directional relations among objects.
- Understand the notion of multiplicity (one-to-one, one-to-many, many-to-many).
- Implement instance-level relationships among objects (association).
- Implement class-level relationships (generalization)
- Work with abstract classes.
- Get used to the naming conventions of the Java language.
- Generate documentation using javadoc.